DSA Day 1 Preparation

Learning Data Structures and Algorithms (DSA) using Python and Java

This repository contains my Day 1 notes for learning Data Structures and Algorithms (DSA) using Python and Java. The main goal is to build strong programming basics before starting DSA topics.

1 Topics Covered

- What is Programming?
- What is a Programming Language?
- Why Python and Java are important
- Python vs Java comparison
- Hello World program in Python and Java
- Introduction to Time Complexity
- Big O Notation
- Space Complexity
- Constraints in DSA
- Choosing the correct algorithm using constraints

2 What is Programming?

Programming means giving instructions to a computer. A computer cannot think by itself; it only follows instructions written by humans. Programming is the process of writing step-by-step instructions to solve a problem.

```
10 + 20 = 30
```

3 What is a Programming Language?

A programming language is a language used to communicate with a computer.

Examples: Python, Java, C, C++, JavaScript.

4 Why Python and Java?

Python

- Beginner-friendly and readable
- Minimal syntax and less boilerplate
- Useful in AI, ML, Data Science, Web Development
- Helps focus on logic

```python
print("Hello World")
```

Java

- Write Once, Run Anywhere
- Statically typed and object-oriented
- Strong structure and good performance
- Used in Android, banking systems, enterprise applications

```java
class Main {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

5 Python vs Java

Aspect	Python	Java
Learning Curve	Smooth and fast	Steep at first
Typing	Dynamic	Static
Speed	Llower	Faster
Main Use	AI, Data Science, Scripting, Web Development	Android Apps, Banking Systems, Big Data
Maintenance	Easy for small teams	Better for huge multi-year projects

6 Big O Hierarchy

Complexity	Name	Example / Use Case
O(1)	Constant Time	Accessing an array index
O(log n)	Logarithmic Time	Binary Search
O(n)	Linear Time	Finding maximum in an array
O(n log n)	Linearithmic Time	Merge Sort, Quick Sort
$O(n^2)$	Quadratic Time	Nested loops

12 Constraint Guide

Input Size (n)	Typical Complexity	When It's Used
$n \leq 12$	O(n!)	Used for permutations
$n \leq 25$	$O(2^n)$	Used in backtracking or exhaustive search
$n \leq 500$	$O(n^3)$	Triple nested loops or some DP problems
$n \leq 5000$	$O(n^2)$	Limit for quadratic solutions
$n \leq 10^6$	O(n log n) or O(n)	Efficient solutions required
$n \geq 10^8$	O(log n) or O(1)	Cannot check every element

6 Before Starting DSA

Before learning DSA topics, it is important to understand Time Complexity, Space Complexity, and Constraints. These concepts help us write efficient solutions.

7 Time Complexity

Time complexity describes how the number of operations grows as the input size increases. We use Asymptotic Analysis and Big O Notation instead of measuring actual time in seconds.

8 Space Complexity

Space complexity means how much memory an algorithm uses. It includes input space and extra space used by the algorithm. Extra space is also called auxiliary space.

9 In-place Algorithm

An in-place algorithm uses constant extra space. It modifies the input directly without creating extra copies.
Example: Swap operation | Auxiliary space: O(1)

10 Recursive Stack Space

Recursion also uses memory. Every time a function calls itself, it is stored in the call stack. If a recursive function goes n levels deep, then its space complexity is O(n).

11 Constraints

Constraints tell us the input size and help decide which time complexity is acceptable. A common rule in competitive programming is that most online judges can handle around 10^{8} operations per second.

13 Day 1 Summary

1. Read the constraints first.
2. Estimate the Big O of your idea.
3. Check whether the number of operations exceeds 10^{8}.
4. Code only if the approach passes the math.

14 Repository Goal

The goal of this repository is to learn DSA step by step using both Python and Java. It will contain DSA theory notes, Python code examples, Java code examples, practice problems, and time/space complexity analysis.

15 Learning Journey

Day 1: Programming Basics, Python vs Java, Time Complexity, Space Complexity, Constraints

Created as part of my DSA preparation journey using Python and Java.

DAY-1 04/05/2026.

1) What is Programming?

→ Programming means giving instructions to a computer.

→ A computer cannot think by itself. It only follows instruction written by humans.

for eg:

10 + 20 = 30

The calculator gives the answer because it is programmed to add numbers.

→ So, programming is the process of writting step-by-step instructions to solve a problem.

→ A programming language is a language used to communicate with a computer.

→ Just like humans use English, Hindi, Telugu, etc.

→ Programmers use languages like

→ Python
→ Java
→ C
→ C++
→ JavaScript.

→ In this series we will cover Python and Java at a time.

→ Why Python and Java are most important:

→ Python: Python is built on the principle: "Readability counts". It is designed to look like English, which minimize the cognitive load on the programmer

Why it's a Powerhouse:

→ Minimalist syntax: Python removes the "boilerplate" code.

→ means the repetitive setup code. You don't have to worry about defining memory types (or) complex structures. Just to print a line.

→ The king of Data & AI: Python is the undisputed leader in Data Science, Machine Learning and Artificial Intelligence. Libraries like Panda, Tensorflow and Pytorch make complex mathematical operations accessible.

→ Prototyping speed: Because you write fewer lines of code, you can build a "Minimum Viable Product" (MVP) than in almost any other language.

→ Learning python teaches you logic before syntax. you focus on what the program should do rather than how the computer handles the memory, making it an ideal "first language".

→ Java: Java follows the principle: "Write Once, Run Anywhere" (WORA). It is a statically typed, object-oriented language that forces you to be disciplined.

Why it's a powerhouse:

→ strict structure: Java is "verbose", but that wordiness is actually a safety net. It requires you to define exactly what kind of data you are using, which prevents bugs in massive system.

→ scalability & Performance: Java is generally faster than Python because it is ~~Completed~~ Compiled into bytecodes and optimized by the Java Virtual Machine (JVM). This is why it's the backbone of big thanks and huge corporation.

→ Object oriented Mastery: Java forces you to learn object oriented programming (OOP) deeply. Concepts like Inheritance, Encapsulation and polymorphism & DNA of Java.

→ Learning Java teaches you how software is built. It forces you to understand how data moves through a system. If you can master Java, learning almost any other C-style language (like C++, C# or JavaScript) becomes significantly easier.

→ Java vs Python

Feature	Python	Java.
Learning Curve	Smooth and fast	Steep at first.
Typing	Dynamic	Static (sometimes Dynamic).
Speed	Slower.	Faster.
Main Use	AI, Data Science, Scripting, Web Dev	Android Apps, Banking Systems, big data.
Maintenance	Easy for small teams	Better for huge, multi-year projects

Eg:

Python:

```
print("Hello World")
```

Java:

```
Class Main {
    public static void main (String [] args) {
        System.out.println("Hello world");
    }
}
```

→ Now we will cover DSA topics in both languages.

→ Before that we need to know about time complexity, space complexity and constraints

① Time Complexity: The 'Growth' Mindset.

As you noted, we don't measure time in seconds because a supercomputer and a 10-year-old laptop will give different results for the same code. Instead, we use Asymptotic Analysis (Big O notation) to describe how the number of operations grows relative to the Input Size(n).

→ The Big O Hierarchy (from fastest to slowest)

→ O(1) → constant time: the speed never changes regardless of data size.

Example: Accessing an index is an array

→ O(logn): Logarithmic Time: The problem size is halved each step.

Example: Binary search. This is extremely efficient for massive datasets.

→ O(n): Linear Time: You touch every element exactly once.

Example: finding the maximum value in an unsorted list.

→ O(nlogn): Linearithmic Time: Often found in efficient sorting algorithms.

Example: Merge sort (or) Quicksort. This is usually the "gold standard" for large scale sorting.

→ O(n^2): Quadratic Time: Nested loops.

Example: Comparing every element to every other element. If $n = 10^5$, $n^2 = 10^{10}$, which will crash most competitive programming Judges.

② Space Complexity: The Memory Budget.

Space Complexity is often overlooked, but in system design (or) Embedded System (like the sensor fusion work) it is critical. It is the sum of Auxiliary Space (extra space used by the algorithm) & the space taken by the input.

→ In-place algorithm: These have O(1) auxiliary space. They modify the input directly creating copies.

Eg: Swap operation.

→ Recursive stack space: Beginners often forget that recursion uses space! Every time a function calls itself, it stays in the "callstack". A recursive function that goes n levels deep has O(n) space complexity.

③ Constraints: Secret Map"

→ The 10^8 rule:

- most modern online judges can handle roughly 10^8 (100million) operations per second. You can use this to "reverse engineer"

Input Size	Target Complexity	Why?
$n \leq 12$	$O(n!)$	Perfect for permutations (Eg: Travelling Salesman)
$n \leq 25$	$O(2^n)$	Common for: backtracking (or) Exhaustive search.
$n \leq 500$	$O(n^3)$	Often involves triples. nested loops (or) certain Dynamic programming.
$n \leq 5,000$	$O(n^2)$	The limit for quadratic solutions.
$n \leq 10^6$	$O(n \log n)$ (or) $O(n)$	This is the most common range. You must be Efficient
$n \geq 10^8$	$O(\log n)$ (or) $O(1)$	You can't even look at every element. Think Binary search (or) Math formulas.

Summary:

1. Read the constraints first.
2. Estimate the Big O of your idea.
3. Calculate: (will n plugged into my Big O exceed 10^8?)
4. Code only if it passes the math.